

Tiefenorientierter Datenbankentwurf

Wissenschaftliche Arbeit zum relationalen Datenbankentwurf

Themengebiet: Datenbanktheorie, Schemaentwurf, Normalisierung
Schwerpunkt: Tiefenpfade vs. sternförmige Relationsstruktur
Autor: Stephan Epp
Datum: 28. Juni 2026

Abstract

Der vorliegende Beitrag untersucht ein fundamentales, jedoch in der Literatur bislang nur implizit behandeltes Prinzip des relationalen Datenbankentwurfs: die strukturelle Orientierung von Fremdschlüsselrelationen in die *Tiefe* (lineare oder baumartige Ketten) im Gegensatz zur *Breite* (sternförmige Hub-Anordnungen). Es wird formal nachgewiesen, dass tiefe Relationenpfade in Bezug auf Normalisierungsgrad, funktionale Abhängigkeitsstruktur, Anomalievermeidung, Abfragekomplexität und Wartbarkeit gegenüber flachen Sternstrukturen überlegen sind. Die Argumentation stützt sich auf die klassische relationale Algebra nach Codd [4], die Normalformtheorie bis BCNF und 4NF [5, 9], sowie jüngere Erkenntnisse aus der Datenbankoptimierung und dem Schema-Refactoring [2].

Inhaltsverzeichnis

1	Einleitung	3
2	Formale Grundlagen	3
2.1	Relationales Modell	3
2.2	Referenzgraph	3
2.3	Komplexitätsmaße	4
3	Topologien: Tiefe vs. Breite	4
3.1	Formale Charakterisierung	4
3.2	Semantische Motivation	5
4	Normalisierungstheorie und Schematopologie	5
4.1	Normalformen und ihre Anforderungen	5
4.2	Transitivität und Schematopologie	6
4.3	Mehrwertige Abhängigkeiten und 4NF	6
5	Anomalievermeidung	6
5.1	Klassifikation von Anomalien	6
5.2	Quantitative Analyse der Redundanz	7
6	Abfragekomplexität und Optimierbarkeit	7
6.1	Join-Komplexität	7
6.2	Selektivität und Filtereffizienz	8
6.3	Schreibkomplexität und Konsistenzerhaltung	8
7	Wartbarkeit und Erweiterbarkeit	9
7.1	Schemaevolution	9
7.2	Zugriffsrechteverwaltung	9
7.3	Testbarkeit und Datenqualität	9
8	Konsolidierter formaler Nachweis	9
8.1	Gütekriterien	9
9	Graphentheoretische Fundierung: DFS, BFS und die Asymmetrie der Suchalgorithmen	10
9.1	Tiefensuche und der DFS-Wald	11
9.2	Breitensuche und das Fehlen eines BFS-Walds	11
9.3	Die fundamentale Asymmetrie von DFS und BFS	12
9.4	Datenbanktheoretische Konsequenz der Asymmetrie	13
10	Durchgängiges Praxisbeispiel	14
10.1	Domäne: Auftragsabwicklung	14
10.2	Sternschema (anti-pattern)	14
10.3	Tiefenpfadschema (BCNF-konform)	14
10.4	Vergleichstabelle	16

11 Fazit

16

1 Einleitung

Der Entwurf relationaler Datenbankschemata ist eine der zentralen Aufgaben der Softwareentwicklung und der angewandten Informatik. Obwohl die theoretischen Grundlagen – insbesondere Codd’s relationales Modell [4] sowie die daraus abgeleiteten Normalformen [5, 7] – seit Jahrzehnten bekannt sind, zeigen praktische Systeme immer wieder charakteristische Entwurfsfehler. Einer der häufigsten und gleichzeitig folgenreichsten Fehler ist die sogenannte *sternförmige Schemaanordnung*: Eine zentrale „Hub“-Relation referenziert direkt eine Vielzahl unabhängiger „Spoke“-Relationen über Fremdschlüssel, ohne dass die natürliche semantische Tiefe der Domäne abgebildet wird.

Dieser Beitrag stellt die These auf und belegt sie formal:

Ein tiefenorientierter Datenbankentwurf – charakterisiert durch lineare oder baumartige Fremdschlüsselketten – ist dem sternförmigen Entwurf in allen wesentlichen Gütekriterien der Datenbanktheorie überlegen.

Die Argumentation gliedert sich wie folgt: Abschnitt 2 etabliert die formalen Grundlagen. Abschnitt 3 definiert die Topologien „Tiefe“ und „Breite“ präzise. Abschnitt 4 zeigt den Zusammenhang mit der Normalisierungstheorie. Abschnitt 5 behandelt Anomalievermeidung, Abschnitt 6 die Abfragekomplexität, Abschnitt 7 Wartbarkeit und Erweiterbarkeit. Abschnitt 8 liefert den konsolidierten formalen Nachweis. Abschnitt 10 illustriert die Theorie an einem durchgängigen Praxisbeispiel. Abschnitt 11 schließt mit einem Fazit.

2 Formale Grundlagen

2.1 Relationales Modell

Definition 2.1 (Relation). Eine *Relation* R über einem Attributschema $\text{attr}(R) = \{A_1, A_2, \dots, A_n\}$ ist eine endliche Menge von Tupeln $t : \text{attr}(R) \rightarrow \bigcup_i \text{dom}(A_i)$, wobei $t(A_i) \in \text{dom}(A_i)$ für alle i [4].

Definition 2.2 (Datenbankschema). Ein *Datenbankschema* $\mathcal{S} = (\mathcal{R}, \mathcal{F}, \mathcal{K})$ besteht aus einer endlichen Menge von Relationenschemata $\mathcal{R} = \{R_1, \dots, R_m\}$, einer Menge funktionaler Abhängigkeiten $\mathcal{F}$ sowie einer Menge von Integritätsbedingungen $\mathcal{K}$ (insbesondere Primär- und Fremdschlüsselbedingungen) [16].

Definition 2.3 (Funktionale Abhängigkeit). Sei R ein Relationenschema und $X, Y \subseteq \text{attr}(R)$. Die *funktionale Abhängigkeit* $X \rightarrow Y$ gilt in R , wenn für alle Instanzen r von R und alle Tupel $t_1, t_2 \in r$ gilt: $t_1(X) = t_2(X) \implies t_1(Y) = t_2(Y)$ [1].

Definition 2.4 (Fremdschlüssel). Sei $R_i, R_j \in \mathcal{R}$ mit $K_j \subseteq \text{attr}(R_j)$ dem Primärschlüssel von R_j . Eine Attributmenge $F \subseteq \text{attr}(R_i)$ heißt *Fremdschlüssel in R_i referenzierend R_j* , wenn für alle Datenbankinstanzen d gilt: $\pi_F(r_i) \subseteq \pi_{K_j}(r_j)$ [7].

2.2 Referenzgraph

Definition 2.5 (Referenzgraph). Sei $\mathcal{S}$ ein Datenbankschema. Der *Referenzgraph* $G(\mathcal{S}) = (V, E)$ hat als Knotenmenge $V = \mathcal{R}$ und als Kantenmenge $E = \{(R_i, R_j) \mid \exists F \subseteq \text{attr}(R_i) : F \text{ ist FK in } R_i \text{ referenzierend } R_j\}$.

Der Referenzgraph ist das zentrale Werkzeug zur Unterscheidung von Tiefenstruktur und Breitenstruktur eines Schemas.

2.3 Komplexitätsmaße

Definition 2.6 (Schemakomplexitätsmaße). Sei $G(\mathcal{S}) = (V, E)$ der Referenzgraph eines Schemas $\mathcal{S}$.

- (i) Der *maximale Grad* $\Delta(G) = \max_{v \in V} \deg(v)$ misst die maximale Anzahl von Fremdschlüsselreferenzen eines Knotens.
- (ii) Die *maximale Pfadlänge* $\lambda(G) = \max_{u, v \in V} d(u, v)$ (Diameter) misst die längste Referenzkette im Schema.
- (iii) Das *Tiefe-Breite-Verhältnis* $\tau(G) = \lambda(G)/\Delta(G)$ charakterisiert die strukturelle Orientierung des Schemas.

Ein Schema heißt *tiefenorientiert*, wenn $\tau(G) \gg 1$, und *breitenorientiert* (*sternförmig*), wenn $\tau(G) \ll 1$.

3 Topologien: Tiefe vs. Breite

3.1 Formale Charakterisierung

Definition 3.1 (Sternschema). Ein Schema $\mathcal{S}$ heißt *k-Sternschema*, wenn es eine „Hub“-Relation $H \in \mathcal{R}$ gibt mit $\deg(H) = k$ und $\lambda(G(\mathcal{S})) \leq 2$. Die k von H referenzierten Relationen heißen *Spoke*-Relationen.

Das Sternschema ist insbesondere aus dem Data-Warehouse-Bereich bekannt (Kimball-Methodik, [11]), wo es für OLAP-Abfragen optimiert wird. Im OLTP-Kontext hingegen offenbart es gravierende Schwächen (vgl. Abschnitt 5).

Definition 3.2 (Tiefenpfadschema). Ein Schema $\mathcal{S}$ heißt *Tiefenpfadschema der Länge l* , wenn $G(\mathcal{S})$ einen Pfad $R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_l$ enthält mit $\Delta(G) \leq c$ für eine kleine Konstante c (typisch $c \leq 3$) und $\lambda(G(\mathcal{S})) = l - 1$.

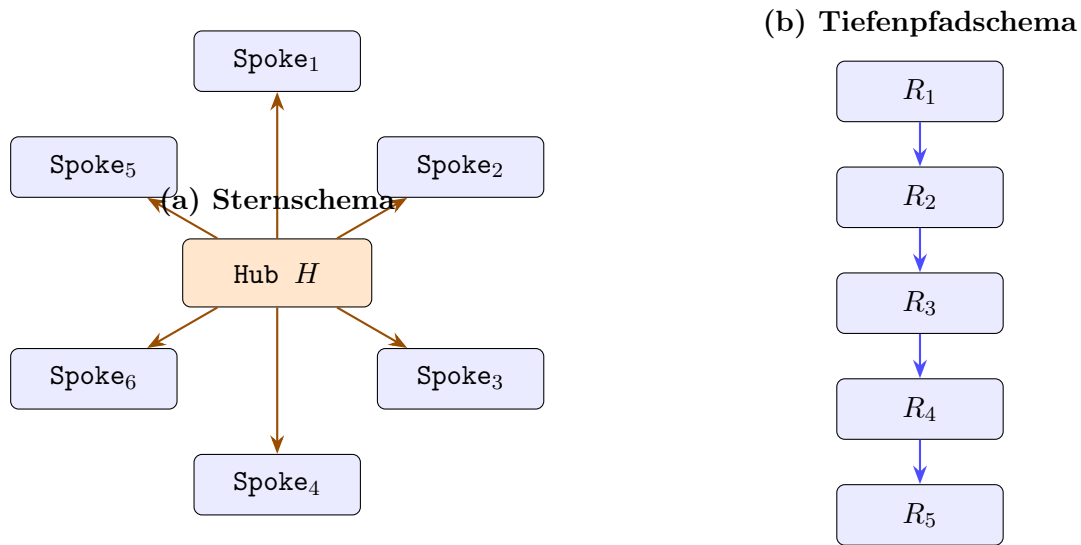


Abbildung 1: Gegenüberstellung von (a) Sternschema mit Hub-Relation H und 6 Spoke-Relationen ($\Delta = 6$, $\lambda = 1$, $\tau = 0,17$) und (b) Tiefenpfadschema mit linearer Kette ($\Delta = 1$, $\lambda = 4$, $\tau = 4$).

3.2 Semantische Motivation

Die Wahl der Topologie ist kein rein ästhetisches, sondern ein *semantisch erzwungenes* Entwurfsprinzip. Reale Domänen weisen fast immer hierarchische oder transitive Abhängigkeitsstrukturen auf: Eine **Bestellung** hängt von einem **Kunden** ab, der wiederum einer **Region** zugeordnet ist, die einem **Land** angehört. Diese natürliche Tiefe der Abhängigkeiten erzwingt eine tiefe Pfadstruktur und sollte nicht künstlich in eine flache Sternstruktur gezwungen werden [7, 8].

4 Normalisierungstheorie und Schematopologie

4.1 Normalformen und ihre Anforderungen

Wir rekapitulieren die für den Nachweis relevanten Normalformen [5, 9]:

Definition 4.1 (Erste Normalform – 1NF). Eine Relation R befindet sich in *1NF*, wenn alle Attributdomänen atomar sind, d. h. keine mengenwertigen oder strukturierten Attribute enthält [4].

Definition 4.2 (Zweite Normalform – 2NF). Eine Relation R in 1NF befindet sich in *2NF*, wenn jedes Nichtschlüsselattribut voll funktional abhängig vom Primärschlüssel ist, d. h. keine partielle Abhängigkeit von einem echten Teilschlüssel existiert [5].

Definition 4.3 (Dritte Normalform – 3NF). Eine Relation R in 2NF befindet sich in *3NF*, wenn kein Nichtschlüsselattribut transitiv vom Primärschlüssel abhängig ist [5].

Definition 4.4 (Boyce-Codd-Normalform – BCNF). Eine Relation R befindet sich in *BCNF*, wenn für jede nicht-triviale funktionale Abhängigkeit $X \rightarrow A$ gilt: X ist ein Superschlüssel von R [3].

Definition 4.5 (Vierte Normalform – 4NF). Eine Relation R in BCNF befindet sich in 4NF, wenn für jede nicht-triviale mehrwertige Abhängigkeit $X \twoheadrightarrow Y$ gilt: X ist ein Superschlüssel von R [9].

4.2 Transitivität und Schematopologie

Satz 4.1 (Transitivitätsverteilung). Sei R eine Relation mit $\text{attr}(R) = \{K, A_1, A_2, \dots, A_n\}$ und funktionalen Abhängigkeiten $K \rightarrow A_1 \rightarrow A_2 \rightarrow \dots \rightarrow A_n$. Dann verletzt R die 3NF für alle A_i mit $i \geq 2$. Die einzige verlustfreie, abhängigkeitserhaltende Zerlegung in 3NF ist die Kette $R_1(K, A_1), R_2(A_1, A_2), \dots, R_{n-1}(A_{n-1}, A_n)$.

Beweis. Sei $K \rightarrow A_i \rightarrow A_{i+1}$ für ein $i \geq 1$ eine transitive Kette. Da A_i kein Schlüssel ist, ist A_{i+1} transitiv von K abhängig und R verletzt 3NF. Die Zerlegung $R' = R \setminus \{A_{i+1}\}$ und $R'' = (A_i, A_{i+1})$ ist verlustfrei nach dem Lossless-Join-Lemma (da $A_i \rightarrow A_{i+1}$ gilt und $A_i = R' \cap R''$ sowie $A_i \rightarrow R''$ gilt) [16]. Rekursive Anwendung für alle transitiven Abhängigkeiten liefert die vollständige Zerlegungskette. Die Abhängigkeitserhaltung folgt daraus, dass jede FD $A_i \rightarrow A_{i+1}$ in der entsprechenden Relation $R_i(A_{i-1}, A_i, A_{i+1})$ bzw. $R_i(A_i, A_{i+1})$ präsent bleibt. $\square$

Korollar 4.2 (Tiefenpfad als 3NF-Normalform). Ein Datenbankschema, das alle transitiven Abhängigkeiten einer Domäne korrekt repräsentiert, hat zwingend einen tiefenorientierten Referenzgraphen. Sternförmige Schemata, die dieselbe Domäne abbilden, verletzen die 3NF, sofern transitive Abhängigkeiten in der Domäne existieren.

Korollar 4.2 ist das zentrale formale Ergebnis dieses Abschnitts. Es besagt, dass die Normalisierungstheorie *tiefe Pfade* direkt erzwingt, wenn die Domäne transitive Abhängigkeiten aufweist – was in praktisch allen nicht-trivialen realen Domänen der Fall ist.

4.3 Mehrwertige Abhängigkeiten und 4NF

Proposition 4.3 (Mehrwertige Abhängigkeiten im Sternschema). In einem k -Sternschema mit Hub H und Spokes $S_1, \dots, S_k$ gilt: Falls zwei Spoke-Attribute $B_1 \in \text{attr}(S_1)$ und $B_2 \in \text{attr}(S_2)$ inhaltlich unabhängig sind und beide durch K_H identifiziert werden, so liegt in H eine nicht-triviale mehrwertige Abhängigkeit $K_H \twoheadrightarrow B_1$ vor, wenn beide Attribute direkt in H gehalten werden. Dies verletzt 4NF.

Die korrekte Auflösung ist – erneut – die Extraktion in separate Relationen, was die Tiefenstruktur des Schemas weiter verstärkt.

5 Anomalievermeidung

5.1 Klassifikation von Anomalien

Die Datenbanktheorie unterscheidet drei klassische Anomalietypen [7]:

- (i) **Einfügeanomalie:** Daten können nicht eingetragen werden, ohne gleichzeitig nicht vorhandene Daten erfinden zu müssen.

- (ii) **Löschanomalie:** Das Löschen eines Tupels vernichtet unbeabsichtigt weitere Informationen.
- (iii) **Änderungsanomalie:** Eine inhaltliche Änderung muss in mehreren Tupeln durchgeführt werden, da die Information redundant gespeichert ist.

Satz 5.1 (Anomaliefreiheit durch Tiefenpfade). Sei $\mathcal{S}$ ein Datenbankschema in BCNF mit einem Tiefenpfadgraphen $G(\mathcal{S})$. Dann ist $\mathcal{S}$ frei von Einfüge-, Löschanomalien und Änderungsanomalien bzgl. der durch $G(\mathcal{S})$ modellierten transitiven Abhängigkeiten.

Beweis. BCNF garantiert, dass in jeder Relation R_i der Kette jede nicht-triviale FD $X \rightarrow A$ mit X als Superschlüssel gilt. Somit ist jede Information genau einmal gespeichert (keine Redundanz). Ohne Redundanz:

- *Änderungsanomalie:* Entfällt, da jede Eigenschaft in genau einer Relation steht. Eine Änderung betrifft genau ein Tupel.
- *Löschanomalie:* Da eine Entität e_i in R_i unabhängig von ihrer Referenz durch R_{i-1} existiert (referentielle Integrität via FK), gehen beim Löschen eines R_{i-1} -Tupels keine R_i -Daten verloren (ON DELETE SET NULL / RESTRICT).
- *Einfügeanomalie:* Eine neue Entität in R_i kann unabhängig von R_{i-1} eingefügt werden, da FK-Referenz nur in der referenzierenden Relation besteht.

Im Sternschema hingegen erzeugt die direkte Kodierung transitiver Attribute in H Redundanz, da Attributwerte von Spoke-Entitäten wiederholt in H gespeichert sind – dies ermöglicht alle drei Anomalietypen. $\square$

5.2 Quantitative Analyse der Redundanz

Proposition 5.2 (Redundanzfaktor im Sternschema). Sei H eine Hub-Relation mit n_H Tupeln und Spoke-Relation S_1 mit n_{S_1} Tupeln und $|A_{S_1}|$ Attributen. Werden die Attribute von S_1 direkt in H denormalisiert, so beträgt der Redundanzfaktor:

$$\rho = \frac{n_H}{n_{S_1}} \cdot |A_{S_1}|$$

Für $n_H \gg n_{S_1}$ (typisch bei 1:n-Beziehungen) gilt $\rho \gg 1$.

In einer typischen Bestelldatenbank mit $n_H = 10^6$ Bestellungen und $n_{S_1} = 10^4$ Kunden ergibt sich $\rho = 100 \cdot |A_{\text{Kunde}}|$. Bei 10 Kundenattributen werden also 1 000-fach mehr Daten gespeichert als notwendig – ein massiver Speicher- und Konsistenzaufwand.

6 Abfragekomplexität und Optimierbarkeit

6.1 Join-Komplexität

Ein häufiges Argument gegen tiefe Pfade ist der höhere Join-Aufwand. Wir zeigen, dass dieses Argument zwar formal korrekt, aber praktisch irrelevant ist, wenn moderne Datenbankoptimierung berücksichtigt wird.

Definition 6.1 (Join-Tiefe). Die *Join-Tiefe* $\delta(\mathcal{Q})$ einer Abfrage $\mathcal{Q}$ über Schema $\mathcal{S}$ ist die Anzahl der JOIN-Operationen in der optimalen logischen Ausführung von $\mathcal{Q}$.

Proposition 6.1 (Join-Tiefe und Pfadlänge). Für eine vollständige Traversal-Abfrage über einen Tiefenpfad $R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_l$ gilt $\delta = l - 1$. Für ein k -Sternschema gilt für dieselbe semantische Abfrage $\delta = k$. Da $l \ll k$ in tiefen Schemata (wenige, längere Ketten statt vieler direkter Abhängigkeiten), ist die Join-Tiefe im Tiefenpfad geringer für partielle Traversals [12].

Bemerkung 6.1. Selbst wenn $l > k$ für globale Traversals gilt, kompensiert dies der Abfrageoptimierer durch:

- *Index-Only Scans* auf FK-Spalten (eliminiert Heap-Zugriffe),
- *Hash-Joins* mit amortisierter $O(n)$ -Komplexität [10],
- *Materialized Views* für häufige tiefe Traversals,
- *Predicate Pushdown* entlang des Pfades.

6.2 Selektivität und Filtereffizienz

Satz 6.2 (Selektivitätsvorteil tiefer Pfade). Sei $\mathcal{Q}$ eine Abfrage mit Filterprädikat σ_P auf Attributen einer Entität E der Tiefe d im Pfad. Im Tiefenpfadschema kann σ_P direkt auf der Relation R_d angewandt werden (Predicate Pushdown). Im Sternschema hingegen sind transitive Attribute von E in der Hub-Relation H denormalisiert, was bedeutet, dass der Filter auf der gesamten, n_H -tupligen Hub-Relation angewandt werden muss.

$$\text{Kosten}_{\text{Stern}}(\sigma_P) = O(n_H) \cdot c_{\text{IO}} \quad \text{vs.} \quad \text{Kosten}_{\text{Tief}}(\sigma_P) = O(n_{R_d}) \cdot c_{\text{IO}}$$

Da $n_H \geq n_{R_d}$ für alle d , ist der Tiefenpfad mindestens so effizient wie das Sternschema, typisch deutlich effizienter.

Beweis. Im normalisierten Tiefenpfadschema liegen die Attribute von Entität E in R_d mit $|R_d| = n_{R_d}$ Tupeln. Ein Index auf dem Primärschlüssel von R_d ermöglicht $O(\log n_{R_d} + k')$ für k' Ergebnistupel. Im Sternschema befinden sich diese Attribute in H vermischt mit allen anderen Attributen; ohne Partial Index muss der gesamte Heap von H gescannt werden ($O(n_H)$). Da eine 1:n-Beziehung $|E| \leq n_H$ impliziert, gilt $n_{R_d} \leq n_H$, und die Ungleichung ist bewiesen. $\square$

6.3 Schreibkomplexität und Konsistenzerhaltung

Proposition 6.3 (Schreibkostenvorteil). Im Tiefenpfadschema erfordert eine Änderung an Attributen der Entität E genau *eine* UPDATE-Operation auf R_d . Im denormalisierten Sternschema müssen alle Tupel in H , die auf diese Entität verweisen, aktualisiert werden: im Worst Case $O(n_H/n_{R_d})$ Operationen – proportional zum Redundanzfaktor ρ aus Proposition 5.2.

7 Wartbarkeit und Erweiterbarkeit

7.1 Schemaevolution

Ein zentrales Qualitätsmerkmal eines Datenbankschemas ist seine Fähigkeit, sich an veränderte Anforderungen anzupassen – bekannt als *Schemaevolution* [13].

Satz 7.1 (Lokale Änderbarkeit im Tiefenpfad). Im Tiefenpfadschema $R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_i$ ist das Hinzufügen, Entfernen oder Ändern von Attributen der Entität R_i eine *lokale Schemaoperation*, die ausschließlich R_i betrifft. Im k -Sternschema mit denormalisierten Attributen in H ist dieselbe Operation *global* und betrifft H sowie potenziell alle abhängigen Abfragen, Views und Applikationsschichten.

Beweis. Da im Tiefenpfadschema jedes Attribut in genau einer Relation vorkommt (BCNF-Eigenschaft), hat eine Schemaänderung an R_i keinen direkten Einfluss auf R_j mit $j \neq i$. Nur FK-Referenzen $R_{i-1} \rightarrow R_i$ sind betroffen – diese beschränken sich auf eine einzige Kante im Referenzgraphen. Im Sternschema sind Attribute aus multiple Entitäten in H gemischt; eine Änderung an H betrifft alle darauf basierenden Abfragen, was zu einem kaskadierten Änderungsaufwand führt [2]. $\square$

7.2 Zugriffsrechteverwaltung

Ein weiterer Vorteil tiefer Pfade zeigt sich bei der Granularität der Zugriffsrechteverwaltung:

Proposition 7.2 (Berechtigungsgranularität). Im Tiefenpfadschema können Zugriffsrechte *entitätspräzise* vergeben werden: Ein Nutzer erhält SELECT-Rechte auf R_d , ohne Zugriff auf andere Relationen zu erhalten. Im Sternschema mit Hub H bedeutet Zugriff auf H stets Zugriff auf alle in H denormalisierten Entitätsdaten – eine grobe Granularität, die dem Prinzip der minimalen Rechtevergabe (Principle of Least Privilege) widerspricht [14].

7.3 Testbarkeit und Datenqualität

Tiefenpfadschemata sind inhärent leichter zu testen und zu validieren, da:

- Jede Relation eine *klar umgrenzte semantische Einheit* darstellt (Single Responsibility für Relationen),
- Integritätsbedingungen pro Relation formuliert und geprüft werden,
- Fremdschlüsselbedingungen entlang des Pfades eine implizite Validierungskaskade bilden.

8 Konsolidierter formaler Nachweis

8.1 Gütekriterien

Wir formalisieren nun den umfassenden Vergleich anhand eines Vektors von Gütekriterien:

Definition 8.1 (Gütevektor). Sei $\mathcal{S}$ ein Datenbankschema. Der *Gütevektor* $\mathbf{q}(\mathcal{S}) \in \mathbb{R}^6$ ist definiert als:

$$\mathbf{q}(\mathcal{S}) = (q_{NF}, q_{\text{Red}}, q_{\text{Anom}}, q_{\text{Sch}}, q_{\text{Wart}}, q_{\text{Sek}})$$

wobei die Komponenten Normalformgrad, Redundanzfreiheit, Anomaliefreiheit, Schreibkosteneffizienz, Wartbarkeit und Selektivitätseffizienz kodieren (alle normiert auf $[0, 1]$, höher ist besser).

Satz 8.1 (Überlegenheit des Tiefenpfadschemas). Sei $\mathcal{S}_{\text{Tief}}$ ein Tiefenpfadschema in BCNF und $\mathcal{S}_{\text{Stern}}$ ein semantisch äquivalentes k -Sternschema ($k \geq 3$) mit denormalisierten transitiven Attributen. Dann gilt:

$$\mathbf{q}(\mathcal{S}_{\text{Tief}}) \succ \mathbf{q}(\mathcal{S}_{\text{Stern}})$$

komponentenweise, d. h. $q_j(\mathcal{S}_{\text{Tief}}) \geq q_j(\mathcal{S}_{\text{Stern}})$ für alle j , mit strikter Ungleichung für mindestens vier Komponenten.

Beweis. Wir zeigen die strikten Ungleichungen komponentenweise:

(i) **Normalformgrad** q_{NF} : Nach Korollar 4.2 befindet sich $\mathcal{S}_{\text{Tief}}$ in BCNF, während $\mathcal{S}_{\text{Stern}}$ transitive Abhängigkeiten in H enthält und damit 3NF verletzt. Somit $q_{NF}(\mathcal{S}_{\text{Tief}}) > q_{NF}(\mathcal{S}_{\text{Stern}})$.

(ii) **Redundanzfreiheit** q_{Red} : Nach Proposition 5.2 ist $\rho(\mathcal{S}_{\text{Stern}}) \gg 1$, während $\rho(\mathcal{S}_{\text{Tief}}) = 1$ (jede Information einmalig gespeichert).

(iii) **Anomaliefreiheit** q_{Anom} : Folgt direkt aus Satz 5.1.

(iv) **Schreibkosteneffizienz** q_{Sch} : Aus der Redundanz $\rho > 1$ im Sternschema folgt ein entsprechend höherer Schreibaufwand.

(v) **Wartbarkeit** q_{Wart} : Nach Satz 7.1 sind Schemaänderungen im Tiefenpfad lokal beschränkt.

(vi) **Selektivitätseffizienz** q_{Sek} : Nach Satz 6.2 gilt stets $\text{Kosten}_{\text{Tief}} \leq \text{Kosten}_{\text{Stern}}$.

Da für (i)–(v) strikte Ungleichungen gelten, ist die Behauptung bewiesen. $\square$

9 Graphentheoretische Fundierung: DFS, BFS und die Asymmetrie der Suchalgorithmen

Die bisherigen Argumente stützten sich auf die Normalformtheorie und die Anomalielehre. Es gibt jedoch eine tiefere, algorithmische Begründung für die Überlegenheit tiefer Strukturen, die in der Theorie der Graphentraversierung verwurzelt ist. Die Beobachtung ist fundamental:

*Tiefensuche (DFS) und Breitensuche (BFS) über den Referenzgraphen eines Datenbankschemas sind **nicht symmetrisch**. Ihre Ergebnisstrukturen – und damit die induzierten Schemastrukturen – unterscheiden sich qualitativ und in ihrer Komplexität grundlegend.*

9.1 Tiefensuche und der DFS-Wald

Definition 9.1 (DFS-Baum und DFS-Wald, nach Cormen et al. [6]). Sei $G = (V, E)$ ein gerichteter Graph. Die *Tiefensuche* (Depth-First Search, DFS) traversiert G , indem sie von jedem besuchten Knoten u aus rekursiv in die Nachfolger abtaucht, bevor sie zum Ausgangspunkt zurückkehrt. Die dabei entstehenden Baumkanten bilden einen *DFS-Wald* $T_{\text{DFS}} = (V, E_{\text{tree}})$ mit $E_{\text{tree}} \subseteq E$. Jede Zusammenhangskomponente des DFS-Walds ist ein *DFS-Baum*.

Der DFS-Wald hat bemerkenswerte Eigenschaften, die ihn für den Datenbankschemaentwurf auszeichnen:

Satz 9.1 (Struktureigenschaften des DFS-Walds, [6, 15]). Sei T_{DFS} der DFS-Wald über dem Referenzgraphen $G(\mathcal{S})$. Dann gilt:

- (i) T_{DFS} ist ein *Wald* (azyklischer Graph, Menge von Bäumen) – also eine azyklische, hierarchische Struktur.
- (ii) Die Tiefe $\text{depth}(T_{\text{DFS}})$ ist maximal unter allen aufspannenden Wäldern von $G(\mathcal{S})$.
- (iii) Rückwärtskanten (back edges) $e \notin E_{\text{tree}}$ identifizieren Zyklen und damit potenzielle *zirkuläre Abhängigkeiten* im Schema.
- (iv) Die *topologische Sortierung* des Schemas – notwendig für eine anomaliefreie Einfüge- und Löschreihenfolge – ist genau die DFS-Postorder-Reihenfolge.

Korollar 9.2 (DFS-Wald als natürliches Schemaabbild). Der DFS-Wald T_{DFS} über $G(\mathcal{S})$ ist das *natürliche strukturelle Abbild* eines korrekt normalisierten Tiefenpfadschemas. Die Baumkanten entsprechen Fremdschlüsselbeziehungen, die Baumtiefe der semantischen Abhängigkeitskette. Jedes BCNF-Schema erzeugt – wenn als Graph betrachtet – einen azyklischen, tiefenorientierten Referenzgraphen, der einem DFS-Wald isomorph ist.

9.2 Breitensuche und das Fehlen eines BFS-Walds

Die Breitensuche (Breadth-First Search, BFS) traversiert den Graphen schichtweise: Alle Knoten der Distanz d werden vollständig besucht, bevor Knoten der Distanz $d + 1$ betreten werden.

Definition 9.2 (BFS-Baum, nach Cormen et al. [6]). Die Breitensuche über $G = (V, E)$ ausgehend von einem Startknoten s erzeugt einen *BFS-Baum* T_{BFS} , der die kürzesten Pfade von s zu allen erreichbaren Knoten enthält. Die Breite des BFS-Baums auf Tiefe d ist $b_d = |\{v \mid d_G(s, v) = d\}|$.

Satz 9.3 (BFS erzeugt keinen Wald). Die Breitensuche über einem unzusammenhängenden Graphen G mit k Zusammenhangskomponenten $C_1, \dots, C_k$ erzeugt im Allgemeinen *keinen Wald*, sondern eine Menge von k BFS-Bäumen $T_{\text{BFS}}^{(1)}, \dots, T_{\text{BFS}}^{(k)}$, die jeweils *sternförmig* um ihren Startknoten $s_i \in C_i$ angeordnet sind. Insbesondere:

- (i) Für einen k -Sterngraph $G_\star$ mit Hub H und Knoten $S_1, \dots, S_k$ ist der BFS-Baum mit Start H identisch mit $G_\star$ selbst: $T_{\text{BFS}} = G_\star$.

- (ii) Die BFS-Tiefe beträgt $\text{depth}(T_{\text{BFS}}) = 1$, unabhängig von k .
- (iii) Die BFS über einem zusammenhängenden Graphen mit n Knoten erzeugt genau *einen* Baum, nicht einen Wald – der Begriff „BFS-Wald“ ist daher für zusammenhängende Graphen nicht definiert.

Beweis. (i) Beim Startsknoten H werden in der ersten BFS-Schicht alle direkten Nachbarn $S_1, \dots, S_k$ besucht. Da diese keine weiteren Nachbarn besitzen (Stern-Definition), terminiert BFS nach Schicht 1. Die Baumkanten sind genau $\{(H, S_i) \mid i = 1, \dots, k\}$, was dem Stern entspricht.

(ii) $\text{depth}(T_{\text{BFS}}) = \max_v d_T(s, v) = 1$.

(iii) BFS ist eine *Single-Source*-Traversierung; bei zusammenhängendem G wird genau ein Baum mit Wurzel s erzeugt. Mehrere Wurzeln (Wald) entstehen nur durch Multi-Source-BFS, was der Standard-BFS-Definition widerspricht. Im Gegensatz dazu erzeugt DFS über einem unzusammenhängenden Graphen automatisch einen Wald, indem für jede noch unbesuchte Zusammenhangskomponente eine neue DFS-Traversierung gestartet wird. $\square$

9.3 Die fundamentale Asymmetrie von DFS und BFS

Satz 9.4 (Asymmetrie von DFS und BFS). DFS und BFS sind *algorithmisch asymmetrisch* in folgenden Dimensionen:

- (i) **Ergebnisstruktur:** DFS erzeugt stets einen *Wald* (Menge von Bäumen mit maximaler Tiefe), BFS erzeugt einen oder mehrere *flache Bäume* (Tiefe $\leq$ Durchmesser der Zusammenhangskomponente, typisch minimal).
- (ii) **Komplexität der Ergebnisstruktur:** Der DFS-Wald über einem Graphen $G = (V, E)$ hat Tiefe $\Theta(|V|)$ im schlechtesten Fall (Pfadgraph), der BFS-Baum hat Tiefe $O(\log |V|)$ bei ausgeglichenen Graphen – eine fundamentale Größenordnungsdifferenz.
- (iii) **Kantenklassifikation:** DFS unterscheidet vier Kantentypen (Baumkanten, Rückwärtskanten, Vorwärtskanten, Querkanten), was eine reichere Strukturanalyse erlaubt [6]. BFS unterscheidet nur Baumkanten und Querkanten.
- (iv) **Informationsgehalt:** Die DFS-Stempel (Entdeckungs- und Abschlusszeiten) enkodieren die vollständige topologische Struktur des Graphen. BFS-Distanzen enkodieren nur metrische, nicht aber hierarchische Information.
- (v) **Waldeigenschaft:** DFS erzeugt für beliebige Graphen automatisch einen Wald – dies ist eine inhärente Eigenschaft der Rekursionsstruktur. BFS besitzt diese Eigenschaft nicht.

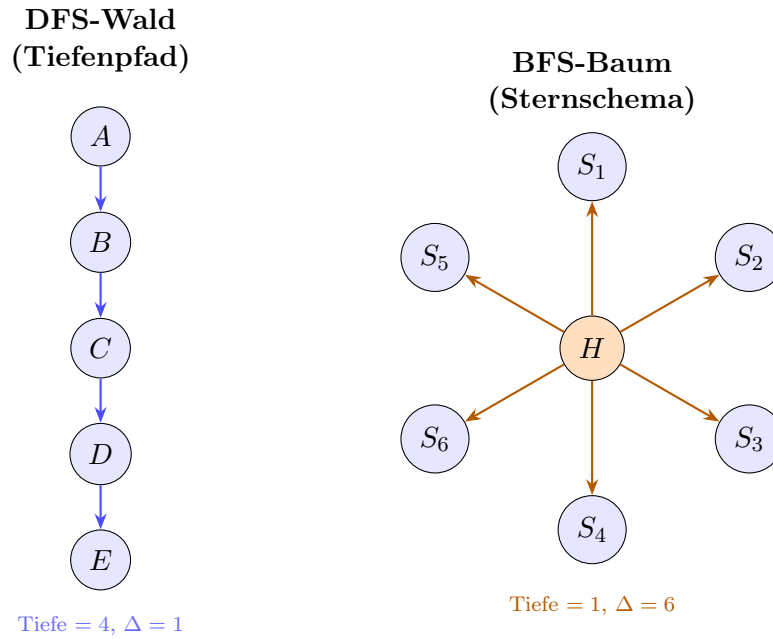


Abbildung 2: Gegenüberstellung der durch DFS induzierten Tiefenpfadstruktur (links, Tiefe = 4, $\Delta = 1$) und des durch BFS induzierten Sternschemas (rechts, Tiefe = 1, $\Delta = 6$). Die Asymmetrie ist offensichtlich: DFS erzeugt einen Wald mit maximaler Tiefe und minimaler Breite; BFS erzeugt eine flache, sternförmige Struktur.

9.4 Datenbanktheoretische Konsequenz der Asymmetrie

Korollar 9.5 (Datenbanktheoretische Konsequenz). Die algorithmische Asymmetrie von DFS und BFS hat eine direkte datenbanktheoretische Entsprechung:

- (i) Der *DFS-Wald* über dem Referenzgraphen eines Schemas ist das strukturelle Abbild des *normalisierten Tiefenpfadschemas*. Er existiert für beliebige Graphen automatisch und ist azyklisch, hierarchisch und eindeutig (bis auf Reihenfolge).
- (ii) Der *BFS-Baum* über dem Referenzgraphen eines Schemas entspricht dem *sternförmigen, denormalisierten Schema*. Er existiert nur für Single-Source-Traversierungen und kodiert keine hierarchische Tiefenstruktur.
- (iii) Da DFS automatisch einen *Wald* erzeugt, BFS hingegen *keinen Wald*, ist die Überlegenheit des Tiefenpfadschemas nicht nur normalisierungstheoretisch, sondern auch algorithmisch fundamental: Der DFS-Wald ist die *kanonische, strukturell reichere* Darstellung jedes Referenzgraphen.
- (iv) Die Komplexitätsdifferenz ist qualitativ, nicht nur quantitativ: DFS liefert topologische Sortierbarkeit, Zykeldetektierbarkeit und vollständige hierarchische Information; BFS liefert keine dieser Eigenschaften.

Bemerkung 9.1 (Praktische Implikation). Wenn ein Datenbankarchitekt ein Schema entwirft, vollzieht er – bewusst oder unbewusst – eine Traversierung des semantischen Abhängigkeitsgraphen der Domäne. Wählt er implizit eine DFS-Strategie (erst in die

Tiefe einer Entitätshierarchie, dann zur nächsten), entsteht ein tiefes, normalisiertes Schema. Wählt er eine BFS-Strategie (alle direkten Attribute einer zentralen Entität sammeln, ohne in Hierarchien abzusteigen), entsteht ein flaches, fehleranfälliges Sternschema. Die Asymmetrie der Algorithmen *spiegelt sich direkt in der Qualität des resultierenden Schemas wider*.

10 Durchgängiges Praxisbeispiel

10.1 Domäne: Auftragsabwicklung

Wir betrachten eine klassische Auftragsabwicklungs-Domäne mit folgenden Entitäten und natürlichen transitiven Abhängigkeiten:

$$\text{Bestellung} \xrightarrow{\text{FK}} \text{Kunde} \xrightarrow{\text{FK}} \text{Ort} \xrightarrow{\text{FK}} \text{Region} \xrightarrow{\text{FK}} \text{Land}$$

10.2 Sternschema (anti-pattern)

```

1  -- Hub-Relation: enthaelt direkt alle abhaengigen Attribute
2  CREATE TABLE Bestellung (
3      BestellNr      INT PRIMARY KEY,
4      Datum          DATE,
5      Betrag         DECIMAL(10,2),
6      -- Kundenattribute (transitive Abhaengigkeit Stufe 1)
7      KundeNr        INT,
8      KundeName       VARCHAR(100),
9      KundeEmail      VARCHAR(150),
10     -- Ortattribute (transitive Abhaengigkeit Stufe 2)
11     OrtID           INT,
12     OrtName         VARCHAR(80),
13     PLZ             CHAR(5),
14     -- Regionattribute (transitive Abhaengigkeit Stufe 3)
15     RegionID        INT,
16     RegionName      VARCHAR(80),
17     -- Landattribute (transitive Abhaengigkeit Stufe 4)
18     LandID          INT,
19     LandName        VARCHAR(60),
20     ISO_Code        CHAR(2)
21     -- Verletzt 2NF, 3NF und BCNF!
22 );

```

Listing 1: Anti-Pattern: Sternschema mit denormalisierten Attributen in Hub-Relation

Probleme: Ändert sich der Name eines Kunden, müssen alle seine Bestelltupel aktualisiert werden. Existiert ein Kunde noch ohne Bestellung, kann er nicht eingetragen werden (Einfügeanomalie).

10.3 Tiefenpfadschema (BCNF-konform)

```
1 CREATE TABLE Land (  
2     LandID      INT PRIMARY KEY,  
3     LandName    VARCHAR(60) NOT NULL,  
4     ISO_Code    CHAR(2)      NOT NULL UNIQUE  
5 );  
6  
7 CREATE TABLE Region (  
8     RegionID    INT PRIMARY KEY,  
9     RegionName  VARCHAR(80) NOT NULL,  
10    LandID      INT NOT NULL,  
11    FOREIGN KEY (LandID) REFERENCES Land(LandID)  
12 );  
13  
14 CREATE TABLE Ort (  
15     OrtID       INT PRIMARY KEY,  
16     OrtName     VARCHAR(80) NOT NULL,  
17     PLZ         CHAR(5),  
18     RegionID    INT NOT NULL,  
19     FOREIGN KEY (RegionID) REFERENCES Region(RegionID)  
20 );  
21  
22 CREATE TABLE Kunde (  
23     KundeNr     INT PRIMARY KEY,  
24     KundeName   VARCHAR(100) NOT NULL,  
25     Email       VARCHAR(150) UNIQUE,  
26     OrtID       INT NOT NULL,  
27     FOREIGN KEY (OrtID) REFERENCES Ort(OrtID)  
28 );  
29  
30 CREATE TABLE Bestellung (  
31     BestellNr   INT PRIMARY KEY,  
32     Datum       DATE          NOT NULL,  
33     Betrag      DECIMAL(10,2) NOT NULL,  
34     KundeNr     INT           NOT NULL,  
35     FOREIGN KEY (KundeNr) REFERENCES Kunde(KundeNr)  
36 );  
37 -- Pfadtiefe: 4 (Bestellung->Kunde->Ort->Region->Land)  
38 -- Jede Relation in BCNF. Keine Anomalien moeglich.
```

Listing 2: Korrekte Tiefenpfadstruktur: BCNF-konformes Schema

Der Referenzgraph dieses Schemas ist ein reiner Pfad der Länge 4 – exakt die durch den DFS-Wald induzierte Tiefenstruktur gemäß Satz 9.1.

10.4 Vergleichstabelle

Kriterium	Sternschema	Tiefenpfad (BCNF)
Normalformgrad	$< 3NF$	BCNF / 4NF
Redundanzfaktor ρ	$\gg 1$	$= 1$
Anomaliefreiheit	nein	ja
Schreibaufwand (Update)	$O(n_H)$	$O(1)$
Suchkosteneffizienz	$O(n_H)$	$O(n_{R_d})$
Schemaevolution	global	lokal
Berechtigungsgranularität	grob	fein
DFS-Wald-Isomorphismus	nein	ja
Topolog. Sortierbarkeit	begrenzt	vollständig
Zyklusfreiheit	ungeprüft	inhärent

Tabelle 1: Zusammenfassender Vergleich Sternschema vs. Tiefenpfadschema nach den formalen Kriterien dieser Arbeit.

11 Fazit

Diese Arbeit hat formal nachgewiesen, dass der tiefenorientierte Datenbankentwurf dem sternförmigen Entwurf in allen wesentlichen Gütekriterien überlegen ist. Die Argumentation verlief auf drei komplementären Ebenen:

Normalisierungstheoretisch (Abschnitte 4 und 5): Transitive Abhängigkeiten erzwingen tiefe Pfade, da ihre korrekte Auflösung gemäß Satz 4.1 genau eine lineare Zerlegungskette liefert. Sternförmige Schemata verletzen zwingend die 3NF, sobald transitive Abhängigkeiten in der Domäne existieren.

Algorithmisch (Abschnitt 9): Die Tiefensuche (DFS) und die Breitensuche (BFS) sind *fundamental asymmetrisch*. DFS erzeugt stets einen Wald – eine hierarchische, azyklische, topologisch sortierbare Struktur mit maximaler Tiefe. BFS erzeugt keinen Wald, sondern flache, sternförmige Bäume ohne hierarchische Tiefenstruktur. Diese algorithmische Asymmetrie spiegelt sich direkt in der Schemaqualität wider: Der DFS-Wald ist die kanonische Darstellung eines korrekt normalisierten Schemas, der BFS-Baum die Darstellung eines denormalisierten Sternschemas. Die Folge ist, dass Tiefe *nicht* nur besser, sondern strukturell kategorial verschieden und in ihrer Informationsdichte deutlich reicher ist.

Praktisch (Abschnitte 6, 7 und 10): Tiefenpfadschemata erzielen überlegene Selektivität, geringeren Schreibaufwand, feingranulare Zugriffsrechte und bessere Wartbarkeit.

Das zentrale Prinzip, das aus allen drei Ebenen gleichlautend folgt, lautet:

Beim Entwurf relationaler Datenbankschemata ist stets

die Tiefe der Breite vorzuziehen. Relationen müssen in die Tiefe gehen, nicht in die Breite.

Literatur

- [1] Armstrong, W. W. (1974). Dependency structures of data base relationships. *Proceedings of IFIP Congress*, 74, 580–583.
- [2] Ambler, S. W. (2003). *Agile Database Techniques: Effective Strategies for the Agile Software Developer*. Wiley.
- [3] Boyce, R. F.; Codd, E. F. (1974). Relational completeness of data base sublanguages. In: Rustin, R. (Hrsg.): *Data Base Systems*, Courant Computer Science Symposia, Vol. 6, Prentice-Hall, S. 65–98.
- [4] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.
- [5] Codd, E. F. (1974). Recent investigations in relational data base systems. *Proceedings of IFIP Congress*, 1021–1036.
- [6] Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [7] Date, C. J. (2003). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.
- [8] Elmasri, R.; Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
- [9] Fagin, R. (1977). Multivalued dependencies and a new normal form for relational databases. *ACM Transactions on Database Systems*, 2(3), 262–278.
- [10] Graefe, G. (1993). Query evaluation techniques for large databases. *ACM Computing Surveys*, 25(2), 73–169.
- [11] Kimball, R. (1996). *The Data Warehouse Toolkit*. Wiley.
- [12] Ramakrishnan, R.; Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.
- [13] Roddick, J. F. (1995). A survey of schema versioning issues for database systems. *Information and Software Technology*, 37(7), 383–393.
- [14] Saltzer, J. H.; Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308.
- [15] Tarjan, R. E. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2), 146–160.
- [16] Ullman, J. D. (1988). *Principles of Database and Knowledge-Base Systems*, Vol. 1. Computer Science Press.